GStreamer

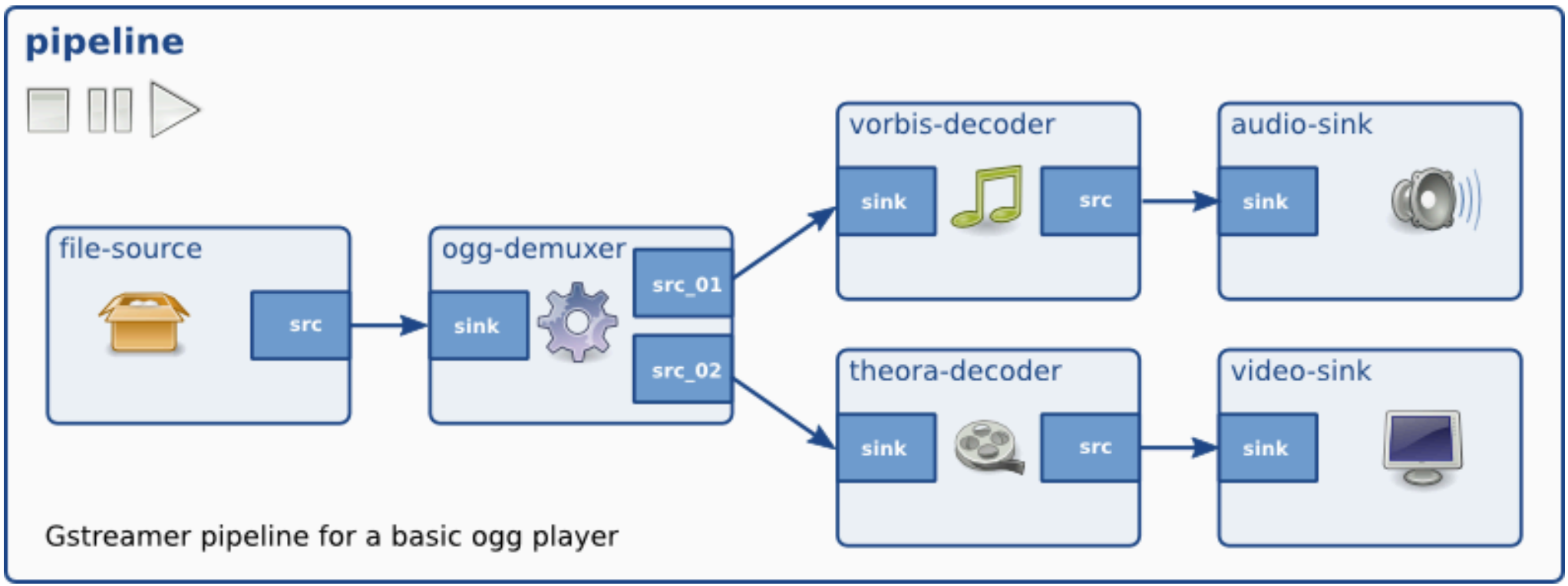
Introduction

- Gstreamer is an open-source multimedia framework that provides a pipeline-based architecture for handling multimedia data such as audio and video.
- https://gstreamer.freedesktop.org/
- GStreamer provides a unified framework for handling various multimedia formats and tasks.
- It supports a wide range of codecs, formats, and protocols.
- Its modular architecture supports plugins and allows the addition of new elements, codecs, and functionality.

Architecture

- `GStreamer` is object oriented, it adheres to the `GObject` model of `GLib 2.0`.
- The main object is an `Element`. Each element has a specific function e.g. reading, writing, encoding or decoding data. By chaining elements, its is possible to create a `pipeline` to achieve a task.
- Elements communicate with each other through `pads`. A pad is a connection point that can be an input (`sink`) or output (`source`). Elements are linked by connecting pads. A pad can restrict the type of data that flows through it. Links are only allowed between two pads when the allowed data types (capabilities) of the two pads are compatible.
- A `bin` is a container for a collection of elements. It can be controlled just like an element
- A `pipeline` is a top level bin. Allowing to control and synchronize all its children.

Example



*E*xample of a GStreamer pipeline

Plugins

- Plugins are selfcontained libraries loaded at runtime.
- All relevant aspects of plugins can be queried at run-time.
- All the properties can be set using the GObject properties, there is no need for header files.
- Core plugins:
  - audiotestsrc, videotestsrc: Generates test audio or video patterns.
  - autoaudiosink, autovideosink: Automatically selects an output and plays audio or displays video.
  - filesrc, filesink: Read from and write to files.
  - decodebin: Automatically selects and configures decoders based on media content.
  - playbin: Automatically plays audio and video from a location

Useful Plugins

alsasink	Sink Audio	Output to a sound card via ALSA
alsasrc	Source Audio	Read from a sound card via ALSA
audioconvert	Filter Converter Audio	Convert audio to different formats
audiodynamic	Filter Effect Audio	Compressor and Expander
audiolatency	Audio Util	Measures the audio latency between the source and the sink
audioloudnorm	Filter Effect Audio	Normalizes perceived loudness of an audio stream
audiomixmatrix	Filter Audio	Mixes a number of input channels into a number of output channels according to a transformation matrix
audioresample	Filter Converter Audio	Resamples audio
clocksync	Generic	Synchronise buffers to the clock
dtmfdetect	Filter Analyzer Audio	This element detects DTMF tones
dtmfsrc	Source Audio	Generates DTMF tones

Useful Plugins

jackaudiosink	Sink Audio	Output audio to a JACK server
jackaudiosrc	Source Audio	Captures audio from a JACK server

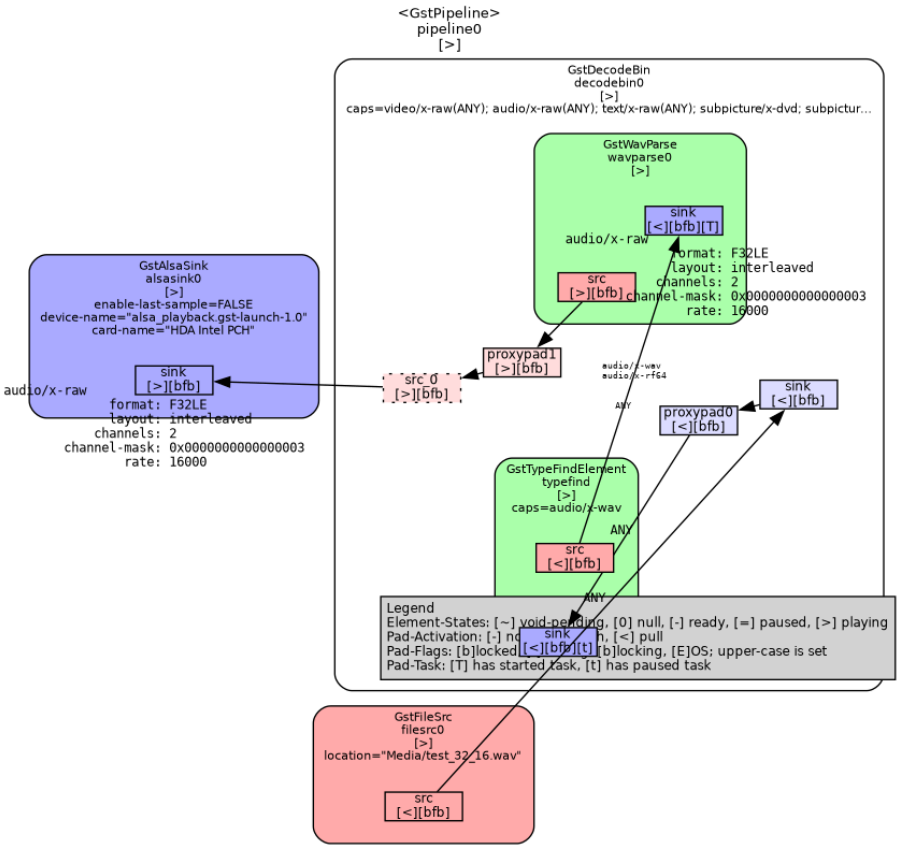
Command line tools

- `gst-inspect-1.0` is a tool that prints out information on GStreamer plugins and elements.
- Without any arguments, it prints a list of all plugins and elements it knows about.
- `gst-launch-1.0` builds and runs a GStreamer pipeline on GStreamer plugins and elements.
- It takes a pipeline description as an argument, this is a list of elements separated by exclamation marks (!). Properties may be appended to elements in the form property=value.
- `gst-launch-1.0` is a tool useful for debugging but shouldn't be used as a standalone application.
- For example, to play an ogg file using ALSA: `gst-launch-1.0 filesrc location=music.ogg ! oggdemux ! vorbisdec ! audioconvert ! audioresample ! alsasink`

Debugging

- `gst-launch-1.0` has a `-v` option to make it verbose
- GStreamer also uses the `GST_DEBUG` environment variable. It takes a debug level from 0 (none) to 9 (memdump). This can also be filtered by element and categories. For example, `GST_DEBUG=2,audiotestsrc:6`, will use level 6 for the `audiotestsrc` element, and 2 for all the others.
- When `GST_DEBUG_DUMP_DOT_DIR` environment variable is set and point to a folder, `gst-launch-1.0` will create a `.dot` file at each state change. `graphviz` can then be used to generate a graph.
  - `gst-launch-1.0 filesrc location=Media/test_32_16.wav ! decodebin ! alsasink`
  - `dot -Kfdp -Tpng -o pipeline.png 0.00.00.021721659-gst-launch.PAUSED_PLAYING.dot`

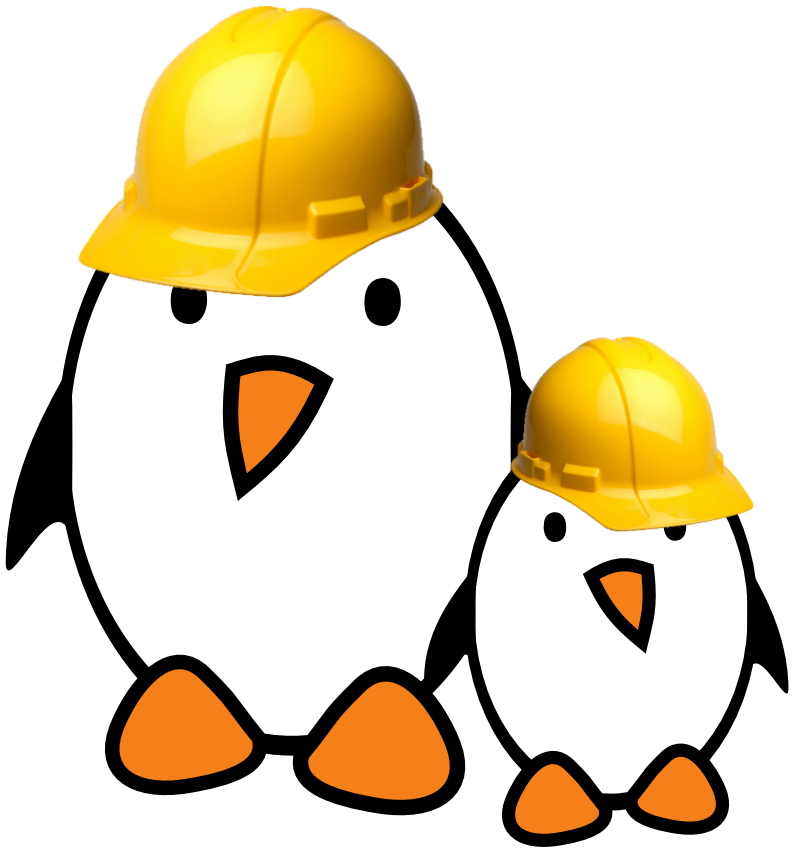
Debugging - graph



Resources

- Documentation: https://gstreamer.freedesktop.org/documentation/. This includes documentation of the API to write application and plugins.
- Plugin list: https://gstreamer.freedesktop.org/documentation/plugins_doc.html

Demo - Gstreamer



- Inspect plugins and elements using `gst-inspect`
- Prepare multiple pipelines with `gst-launch`